## 1. NLP Basics & Challenges

**Goal:** Enable computers to understand/process human language.

### Types of Ambiguity

- **Word Sense:** one word, many meanings. "I saw her *duck*" (animal vs. verb).
- **PP Attachment:** which part modifier attaches to. "I saw her duck *with a telescope*" (telescope with me or her?).
- **Headline/Syntactic:** compressed phrasing. "San Jose cops kill man with knife" (who has knife?).
- **Pronoun/Coref:** unclear pronoun referent. "The umbrella can't fit in the bin because *it* is too big/small" (it = umbrella or bin?).

### Modern Challenges

- **Scalability:** 1M words (Penn Treebank) → billions (Web). Need massive corpora.
- **Dynamic Language:** cyber lingo, slang evolve (LOL, G2G).
- **Hallucination:** LLMs fabricate facts, fail basic reasoning (e.g., count 20 baseballs as 24).

## 2. Coreference & Evaluation

**Coref Resolution:** group mentions (pronouns, common nouns, NEs) referring to same entity. Not all nouns/pronouns are mentions ("*It* is raining" → no referent).

### MUC (Link-based)

Computes P/R on mention **pairs/links**.

$$P = \frac{TP\ pairs}{\text{predicted pairs}},\ R = \frac{TP\ pairs}{\text{gold pairs}}$$

**Flaw:** putting ALL mentions in 1 cluster ⇒ $R = 1$ artificially.

### B-CUBED (Mention-based)

Computes P/R per *individual mention*, then averages. Fixes MUC giant-cluster flaw.

### F1 Aggregation

$$F1 = \frac{2PR}{P+R}$$

- **Macro F1:** compute F1 per doc, then average.
- **Micro F1:** pool all TP/FP/FN across docs first, then single global F1.

## 3. Word Representations & PMI

### One-Hot Vectors

Dim = $|V|$, sparse, orthogonal. $v_{happy} \cdot v_{sad} = 0$ ⇒ no similarity notion.

### Taxonomy (WordNet)

Manual hierarchy of synonyms. Flaws: labor-intensive, misses nuance, can't track new words.

### Distributional Hypothesis

"A word is characterized by the company it keeps." Build co-occurrence matrix $M$; compress via PCA/SVD (low-rank).

### PMI

$$PMI(x,y) = \log_2 \frac{P(x,y)}{P(x)P(y)}$$

Positive ⇒ co-occur more than independent; Negative ⇒ less. **PPMI:** $\max(PMI, 0)$ (drop negatives, often noisy).

**Estimate from counts** (window/doc):

$$P(x) = \frac{\#x}{N},\ P(x,y) = \frac{\#(x,y)}{N}$$

## 4. Word Embeddings (Word2Vec)

### Skip-Gram

Given center word, predict context (outside) words in window.

$$P(o|c) = \frac{\exp(u_o^\top v_c)}{\sum_{w\in V}\exp(u_w^\top v_c)}$$

Loss: $J = -\sum_t \sum_{j\neq 0} \log P(w_{t+j}|w_t)$.

### Parameter Count

Each word has 2 vectors: center $v_w$, context $u_w$. Total parameters $= 2|V|d$.

### Gradient / Update

$$\frac{\partial J}{\partial v_c} = u_o - \sum_w P(w|c)u_w$$

"Observed context minus expected context." Update subtracts expected weighted sum from true $u_o$.

### Properties

- **Analogies** via vector arithmetic: $v_{king} - v_{man} + v_{woman} \approx v_{queen}$.
- **Similarity** via cosine: $\cos(a,b) = \frac{a\cdot b}{\|a\|\|b\|}$.
- **Biases:** embeddings encode societal stereotypes (surgeon→he, nurse→she).

## 5. Language Models (N-grams)

### Chain Rule

$$P(w_1...w_T) = \prod_{t=1}^{T} P(w_t|w_1...w_{t-1})$$

### N-Gram MLE

Bigram: $P(w_i|w_{i-1}) = \frac{C(w_{i-1},w_i)}{C(w_{i-1})}$. Trigram: condition on 2 prev.

### Sparsity & Zipf

Zipf: $r$-th most common word has freq $\propto 1/r$. Unseen n-gram ⇒ P=0 kills whole sequence.

### Smoothing

**Add-One (Laplace):**

$$P = \frac{C(w_{i-1}w_i)+1}{C(w_{i-1})+V}$$

Must add $V$ (vocab size) in denominator so probs sum to 1.

**Add-$\lambda$:**

$$P = \frac{C(w_{i-1}w_i)+\lambda}{C(w_{i-1})+\lambda V}$$

$\lambda \in (0,1)$ tuned on validation; avoids stealing too much mass from seen n-grams.

### Evaluation

**Perplexity:** $PP = P(w_1..w_T)^{-1/T} = \exp(-\frac{1}{T}\sum \log P(w_t|\cdot))$. Lower = better.

## 6. Neural Language Models

### Feedforward LM (Fixed Window)

Concatenate last $n-1$ word embeddings; feed to MLP + softmax.

+ Fixes sparsity (dense reps, generalizes).
– Fixed context size; can't handle variable lengths; no param sharing across positions.

### RNN LM

Same $W_h, W_e$ applied at *every* step.

$$h_t = \sigma(W_h h_{t-1} + W_e x_t),\ \hat{y}_t = \text{softmax}(Uh_t)$$

+ Handles variable lengths; parameter sharing; context grows.
– Sequential (slow); **vanishing gradient** ⇒ forgets far-away words.

### LSTM

Adds cell state $c_t$ with gates:

- **Forget gate** $f_t$: what to drop from $c_{t-1}$.
- **Input gate** $i_t$: what new info to add.
- **Output gate** $o_t$: what to expose as $h_t$.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,\ h_t = o_t \odot \tanh(c_t)$$

Gates explicitly control memory ⇒ mitigates vanishing gradient.

## 7. Seq2Seq & Decoding

### Seq2Seq

Encoder RNN reads source → final hidden = single fixed vector. Decoder RNN generates target conditioned on it. **Bottleneck:** entire source squeezed into one vector ⇒ info loss on long sentences.

### Decoding Algorithms

**Greedy:** at each step pick $\arg\max P(w_t|\cdot)$. Fast; locally optimal, globally suboptimal.

**Beam Search:** keep top-$k$ partial hypotheses (beam width $k$) by cumulative log-prob.

- **If $k = 1$, beam search ≡ greedy.**
- Larger $k$: better search, more compute; too large can hurt fluency.

## 8. Attention

**Motivation:** break Seq2Seq bottleneck by letting decoder look at *all* source hidden states.

### Mechanism (each decoder step $t$)

1. Score: $e_{t,i} = s_t^\top h_i$ (dot) for each source state $h_i$.
2. Softmax: $\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}$.

⇒ **weights** $\alpha \in [0,1]$, $\sum_i \alpha_{t,i} = 1$.

3. Context: $c_t = \sum_i \alpha_{t,i} h_i$.
4. Combine with $s_t$, predict $w_t$.

### Benefits

- Solves bottleneck (direct access to all source states).
- Helps with vanishing gradients (shortcut paths).
- **Interpretable** $\alpha$ learns soft word-alignments.

## 9. Self-Attention & Transformers

Removes RNN recurrence ⇒ **parallel** processing over sequence.

### Q, K, V

Inputs $X \in \mathbb{R}^{L\times D}$. Project:

$$Q = XW^Q,\ K = XW^K,\ V = XW^V$$

Each token *queries* others (via $Q$ vs. $K$) and pulls info from their *values* ($V$).

### Scaled Dot-Product

$$\text{Attn}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

### Space / Compute

$QK^\top$: $(L\times D)(D\times L) = L\times L$ attention matrix.

$$\boxed{\text{Space/Time} = O(L^2)}$$

Scales **quadratically** in sequence length.

**Extras**

- **Multi-head:** $h$ parallel $Q, K, V$ heads $\rightarrow$ concat $\rightarrow$ projection.
- **Positional encodings:** needed since self-attention is order-agnostic.
- **Masking:** decoder masks future positions to preserve autoregressive property.

## 10. Implementation Practice

**HuggingFace Tokenizers**

- `from transformers import AutoTokenizer, AutoModel`
- `tok = AutoTokenizer.from_pretrained("bert-base-uncased")`
- `ids = tok("Hello world", return_tensors="pt")`
- `model = AutoModel.from_pretrained(...)`
- Subword tokenization (BPE / WordPiece) handles OOV.
- Special tokens: `[CLS]`, `[SEP]`, `[PAD]`, `[MASK]`.

**Cloud GPU Commands**

- `ssh user@host` – connect to instance.
- `scp file user@host:path` – copy files.
- `nvidia-smi` – GPU status, memory, processes.
- `tmux` / `screen` – persistent sessions through disconnects.
- `CUDA_VISIBLE_DEVICES=0 python train.py` – pin GPU.
- `pip install torch transformers datasets`.
- `watch -n 1 nvidia-smi` – live monitoring.

## Quick Formulas Cheat

- $PMI = \log_2 \frac{P(x,y)}{P(x)P(y)}$; PPMI $= \max(0, PMI)$.
- W2V params: $2|V|d$. Cosine: $\frac{a \cdot b}{\|a\|\|b\|}$.
- Laplace: $\frac{C+1}{C(\cdot)+V}$. Add-$\lambda$: $\frac{C+\lambda}{C(\cdot)+\lambda V}$.
- Perplexity $= 2^{-\frac{1}{T}\sum \log_2 P}$.
- F1 $= \frac{2PR}{P+R}$.
- Attention softmax sums to 1; self-attn $= O(L^2)$.
- Beam $k$=1 $\Leftrightarrow$ greedy.

## Common Pitfall Checks

- MUC recall = 1 if one giant cluster $\Rightarrow$ use B-CUBED.
- Laplace: denom must include $+V$, not $+1$.
- W2V has TWO vectors per word, not one.
- Attention weights: after softmax, in $[0, 1]$, sum to 1 (NOT raw scores).
- Self-attention is permutation-invariant without positional encodings.
- One-hot dot product = 0 for different words $\Rightarrow$ no similarity.